

CPU and Buses

Putting integrated circuits, clocks, and memory chips together

CPU Chips

All modern CPUs are contained on a single chip, and each of those chips contain a set of pins.

Some of those pins are to *receive* signals to the outside world, and some *accept* signals from the outside world, and some do **both**.

By understanding the function of all the pins, we can learn how the CPU interacts with the memory and I/O devices at the digital logic level.

A simple 8-bit CPU like the Z80 has 40 pins. A modern 64-bit CPU like an Intel Core i7 can have well over 1,000 contacts (via a Land Grid Array socket rather than individual pins).

The *categories* of signals are the same, there are just far more of them, and many are duplicated for power delivery and noise reduction.

CPU Chips

Most pins are not simple wires that are always "on". Many address and data pins are **tri-state**: they can output a `0` , a `1` , or a third *high-impedance* state where they electrically disconnect themselves from the bus.

This matters because a bus is a **shared resource** - if two devices tried to drive it with conflicting values (one `0` , one `1`) at the same time, it could damage the circuitry.

- Tri-stating lets a device "let go" of the bus so another device (or the CPU) can drive it instead.
- This is what makes it possible for many different chips to share the same physical wires without permanently wiring each one directly to every other.

CPU Pins

The CPU pins can be grouped into 3 main categories:

- Address pins,
- Data pins, and
- Control pins

These pins are connected to similar pins on the memory and I/O devices through a set of parallel wires called a **Bus**

CPU Pins

Example

For a CPU to fetch and run an instruction from memory, it needs to:

1. *Put* the memory address of that instruction *on its address pins*
2. *Assert* one or more *control lines* to inform the memory that it wants to read a word
3. The memory *replies* by *putting* the requested word on the *CPU's data pins*
4. The memory *asserts* a signal that it's done
5. The CPU *sees* this *signal* and accepts the word
6. It carries out the instruction
7. This can then repeat for the next instruction

What's important to understand is that the CPU communicates with the memory and I/O devices by presenting and accepting signals on its pins

No other communication is possible

Control pins

In addition to address and data pins, each CPU has some control pins that regulate flow and timing of data.

They all usually have pins for:

1. Power
2. Ground
3. Clocks

But other control pins vary between CPU architectures, though they are usually grouped into:

1. Bus control
2. Interrupts
3. Bus arbitration
4. Coprocessor signaling
5. Status
6. Miscellaneous

Control Pins

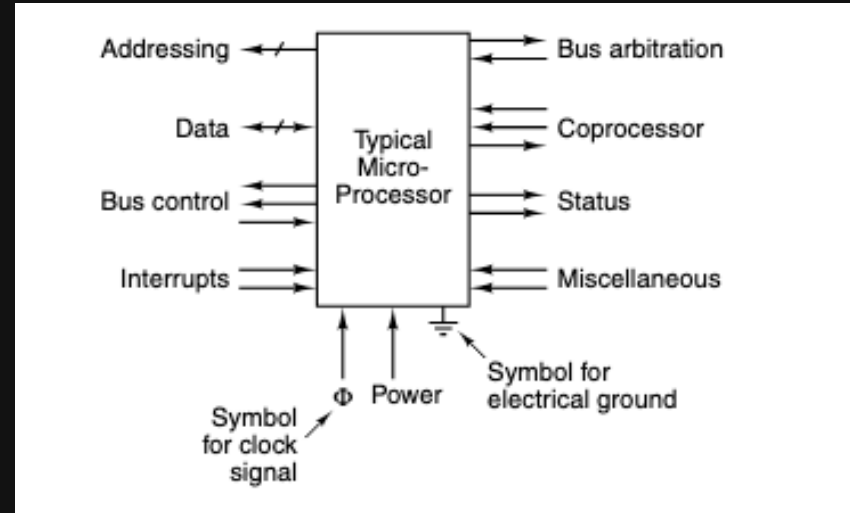
The bus control pins are mostly CPU → bus, for when the CPU wants to read or write

- The interrupt pins are inputs from I/O Devices

The bus arbitration pins are for regulating bus traffic

- Coprocessor pins are for communicating with things like floating point chips and graphics chips

And other miscellaneous pins that some CPUs have



Buses

Buses

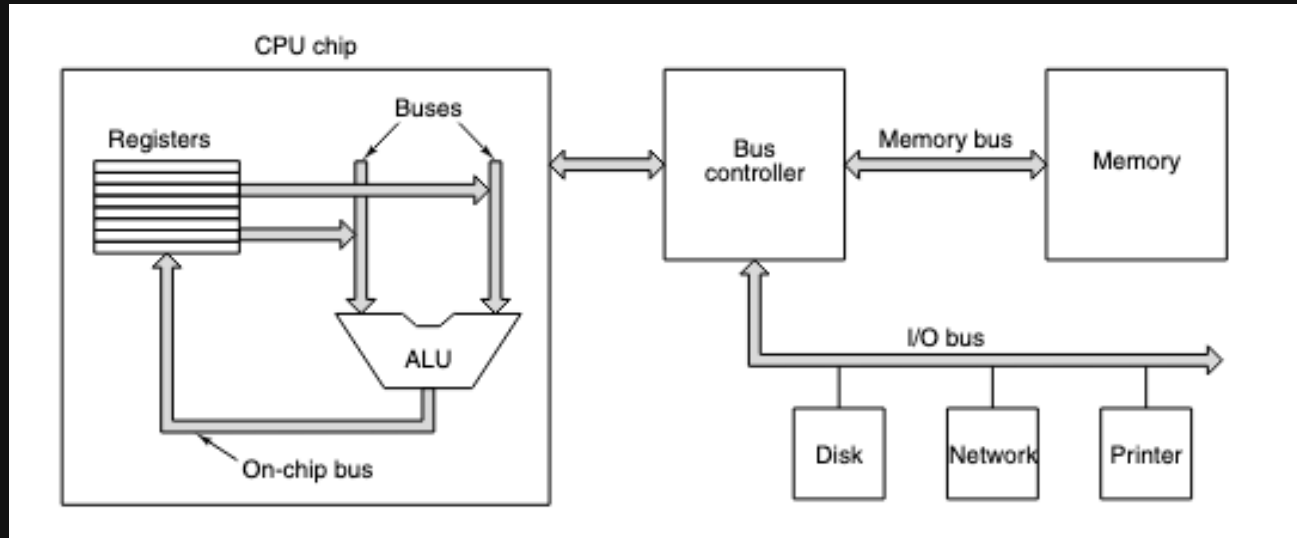
An electrical *pathway* between multiple devices, usually categorized by their function. They can be used internally inside the CPU to transport data to and from the CPU.

Or they can be *external*, connecting to memory or I/O devices.

And each has its own requirements and properties

System Bus

Early computers had a single external bus, usually called a **system bus**. It consisted of 50–100 parallel copper wires etched into the motherboard, with connectors for I/O



Modern PCs usually have special purpose bus' between the CPU and memory, and the CPU and I/O devices.

System Bus

The classic example is the **Omnibus** used in the PDP-8 (1965): a single set of ~ 96 wires that *every* module in the machine plugged into, including the CPU, memory boards, and I/O interfaces.

- **Advantage:** Simple, cheap, and easy to expand.
- **Disadvantage:** Every device shares the *same* bandwidth. As more devices are added, contention increases and overall throughput per device drops.

This is why modern systems split the single system bus into **specialized buses**:

- A high-speed **memory bus** (e.g. DDR channels) between the CPU and RAM, optimized for bandwidth and low latency
- A separate, often slower **I/O bus** (e.g. PCIe) for peripherals, since I/O devices rarely need the same bandwidth as memory

Bus Protocols

Designers of the CPU can use whatever bus they want inside. But for **external** buses, they have to use a bus protocol that is *compatible* with the memory and I/O devices.

These protocols are a set of well-defined rules, both logically and electrically, that all devices on the bus must follow to be able to communicate.

Though some buses, especially for smaller embedded systems, are custom designed for a specific application since they don't need to deal with compatibility.

Bus Protocols

The world would be a better place if all but one disappeared, and we all used the same one. However, that's unlikely to ever happen.

Too much money is involved in the design and manufacture of these systems for companies to give up their proprietary designs.

And backward compatibility limits the ability for new designs to take over the market.

Some popular bus protocols include:

- Omnibus (PDP-8)
- Unibus (PDF-11)
- Multibus (8086)
- VME bus (physics lab equipment)
- IBM PC bus (PC/XT)
- ISA bus (PC/AT)
- EISA bus (80386)
- Microchannel (PS/2)
- Nubus (Macintosh)
- PCI bus (many PCs)
- SCSI bus (many workstations)
- Universal Serial Bus (USB), and
- FireWire (consumer electronics)

Bus Protocols

A rough comparison of how bus bandwidth has grown as protocols evolved:

Bus	Era	Width	Typical Bandwidth
ISA	1980s	16-bit	~8 MB/s
PCI	1990s	32-bit	~133 MB/s
PCIe 1.0 (x16)	2000s	serial, 16 lanes	~4 GB/s
PCIe 5.0 (x16)	2020s	serial, 16 lanes	~64 GB/s

Newer buses moved from wide **parallel** designs (many wires, all switching together)

to narrow, very fast **serial** designs (fewer wires, each running at extremely high frequency).

This avoids the *bus skew* problem discussed later, since there's no need to keep dozens of parallel signals synchronized.

Bus Relationships

How buses work can be described in terms of a Master/Slave relationship.

Active buses (masters) can **initiate** bus transfers whereas passive (slaves) buses wait for requests.

When the CPU orders a disk controller to read or write, the CPU is acting as the master and the disk controller is the slave.

But later, the disk controller may act as the master when it commands the memory to accept the words it is reading from disk

Bus Relationships

A very common case of a non-CPU master is a **DMA controller**.

1. The CPU tells the DMA controller: "move 4KB from disk buffer X to memory address Y", then continues executing other instructions
2. The DMA controller becomes bus master, and directly transfers the data between the disk controller and memory *without* involving the CPU for each word
3. Once finished, the DMA controller raises an interrupt to tell the CPU the transfer is complete

This is exactly why the rule "memory can never be a master" matters: memory must always be a *passive* responder, ready to accept requests from whichever device (CPU or DMA controller) currently holds the bus.

Bus Relationships

There are several combinations of master and slave relationships possible, with the only rule being that memory can never be a master

Master	Slave	Example
CPU	Memory	Fetching instructions and data
CPU	I/O device	Initiating data transfer
CPU	Coprocessor	CPU handing instruction off to coprocessor
I/O device	Memory	DMA (Direct Memory Access)
Coprocessor	CPU	Coprocessor fetching operands from CPU

And usually the binary signals that these devices output are too weak to power a bus, which is why most bus masters are connected to the bus by a circuit called a **bus driver**, which is essentially a digital amplifier.

Similarly, bus slaves are usually connected to the bus by a **bus receiver**

And devices that can do both are connected by a **bus transceiver**

Bus Design Choices

Bus design is complicated enough to have its own field of study.

But for our purposes, we only need to understand a few basic design choices that affect how a bus works

Primarily the `width`, the `clocking`, the `arbitration`, and the `operation`

And each of these have a substantial impact on the *speed* and *bandwidth* of the bus.

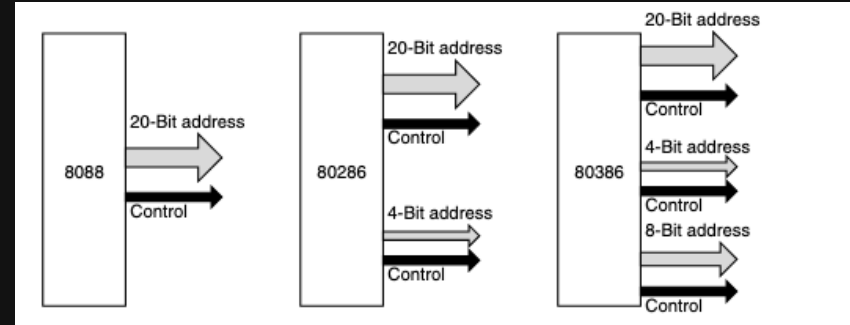
Bus Width

The most obvious design parameter. *More* address lines a bus has, the *more* memory the CPU can address directly.

If a bus has n address lines, it can address 2^n memory locations.

However, larger buses need more wires, which both increase the cost, make it take up more physical space, and require bigger connectors.

A system with a 64-line address bus and 2^{32} bytes of memory will cost more than one with a 32-line address bus and 2^{32} bytes of memory.



Bus Width

Address bus growth in the x86 family

CPU	Address lines	Max addressable memory
8088	20	$2^{20} = 1 \text{ MB}$
80286	24	$2^{24} = 16 \text{ MB}$
80386	32	$2^{32} = 4 \text{ GB}$
x86-64 (modern)	48 (typical)	$2^{48} = 256 \text{ TB}$

Each jump required either widening the physical bus (more pins, more traces on the motherboard) or, eventually, moving away from a simple flat parallel address bus altogether toward memory controllers integrated into the CPU package.

Bus Width

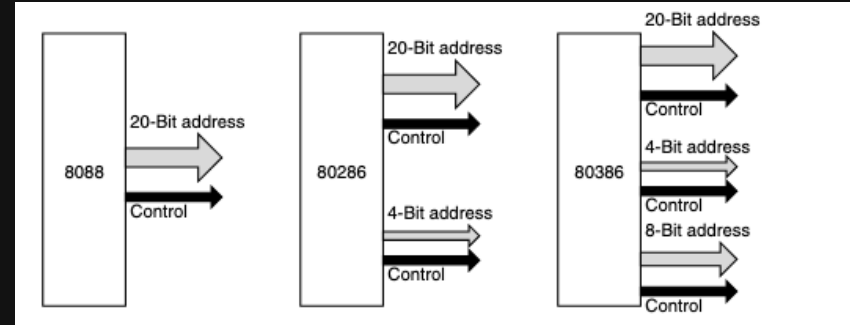
This led to many designs where the CPU had significantly more complicated architectures, due to the designers not initially starting with a wide enough bus.

In this example, the 8088 CPU had a 20-bit address bus for 1 MB of memory,

but the 80286 wanted 16 MB of memory so *four more bus lines* were added.

And because of **backwards compatibility**, the original 20 lines were kept, leading to a more complex design.

And was then repeated once more with the 80386



Bus Width

Not only does the number of address lines grow over time, the number of data lines also tends to grow. Though it grows for different reasons.

There are 2 main ways to increase the bandwidth of a bus:

1. decrease bus *cycle time*, or
2. increase the data *bus width*

The first one is possible (but difficult) and leads to more *bus skew*. An effect caused by signals traveling at different speeds down different wires. The faster the bus, the more skew

Another problem with speeding up the bus is **backward compatibility**. Old boards designed for slower buses will not work with faster ones.

The second one is the usual approach, but usually leads to a *multiplexed bus* because super wide buses are expensive. And so the data lines are shared for both address and data, but at different times

This leads to narrower bus widths (and costs) but a slower system.

Bus Width

Bus bandwidth is roughly:

$$\text{Bandwidth} = \text{data width (bytes)} \times \frac{1}{\text{cycle time}}$$

- A 32-bit (4 byte) bus running at 100 MHz (10 ns cycle time), transferring one word per cycle:

$$4 \text{ bytes} \times 100,000,000 \text{ cycles/s} = 400 \text{ MB/s}$$

- Doubling the data width to 64-bit (8 bytes) at the *same* clock rate doubles bandwidth to 800 MB/s, without needing a faster (and skew-prone) clock
- Alternatively, doubling the clock rate to 200 MHz on the original 32-bit bus also achieves 800 MB/s, but is much harder to engineer reliably because of bus skew and signal integrity issues

This is why real designs (like DDR memory) usually widen the bus *and* moderately increase the clock, rather than pushing either dimension to its limit alone.

Bus Width

The original **PCI bus** multiplexes address and data onto the same 32 physical lines (called `AD0` - `AD31`):

1. During the first bus cycle, the lines carry the **address**
2. During the following cycle(s), the *same* lines carry the **data**

This halves the pin count compared to having separate address and data buses, at the cost of needing at least 2 bus cycles per transfer instead of potentially 1. It's a direct tradeoff between **pin cost** and **raw speed**.

Bus Clocking

Buses can be divided into two distinct categories depending on their clocking

- A synchronous bus has a line driven by a crystal oscillator
- A asynchronous bus has no clock line

Each type runs on bus cycles, and they can be of any length

Bus Clocking

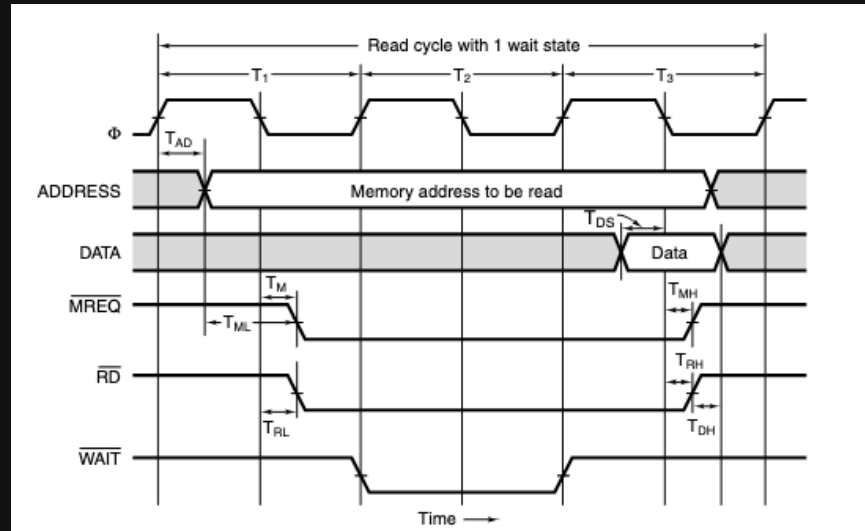
	Synchronous	Asynchronous
Timing reference	Shared clock signal	Handshake signals only
Design complexity	Simpler to design and verify	More complex control logic
Speed with mismatched devices	Limited to whole clock cycles (rounds up)	Adapts exactly to each device's actual speed
Backward compatibility	Hard, old boards can't handle a faster clock	Easier, handshake naturally adapts
Common use	Most PC buses (memory bus, PCI, etc.)	Some legacy minicomputer buses (Unibus), some modern interconnects

Both approaches still need to obey the **speed of light / signal propagation limits** neither can make a signal travel faster down a wire than physics allows, which is why bus length is also a design constraint.

Synchronous

A synchronous bus runs on a crystal that usually runs between 5 — 133 MHz with all activities running in sync with the clock

In our example, we'll be using a 100 MHz clock, which gives a bus cycle time of 10 ns. We'll also assume that reading from memory takes 15 ns from the time the address is stable



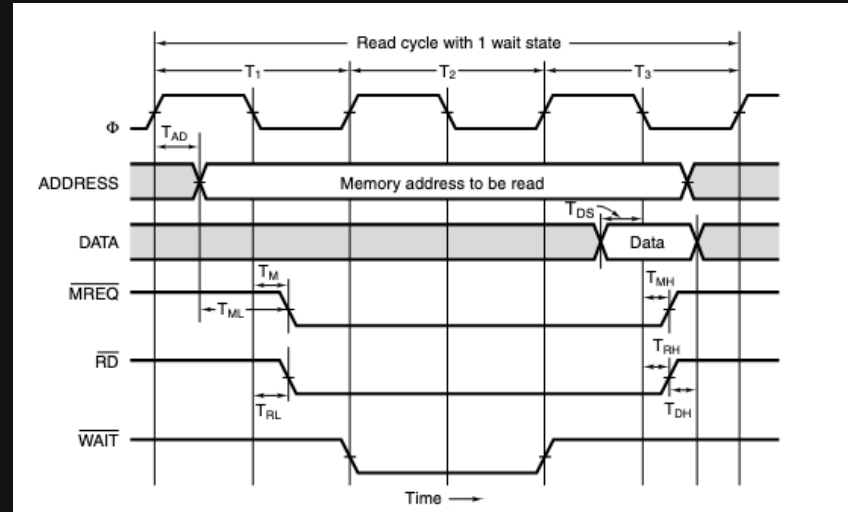
This means that it takes **3 bus cycles** to read a word from memory

Synchronous

In our example (100Mhz clock, 10ns cycle time, 15ns memory read time)

1. The first cycle starts at the rising edge of T_1
2. The third one ends at the rising edge of T_4

Also note that the electrical signals don't change instantaneously, they take some time to propagate down the wires. Assume this takes 1 ns

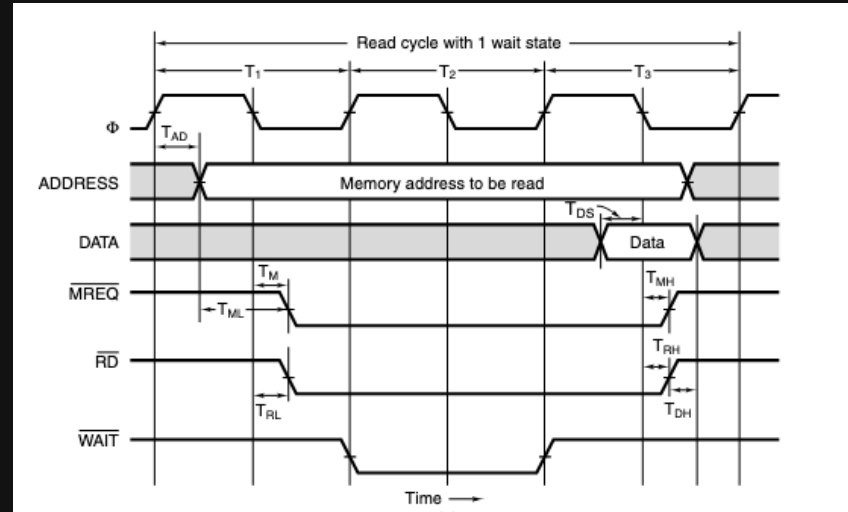


Synchronous

Order of operations in this bus

1. At the start of T_1 , the CPU initiates a memory read
2. During the first half of T_1 , the address the CPU wants to read is placed on the address bus

Note how the data line isn't significant until T_3



Synchronous

Order of operations in this bus

3. After the address line is stable, $\overline{MREQ}$ and $\overline{RD}$ are **asserted**

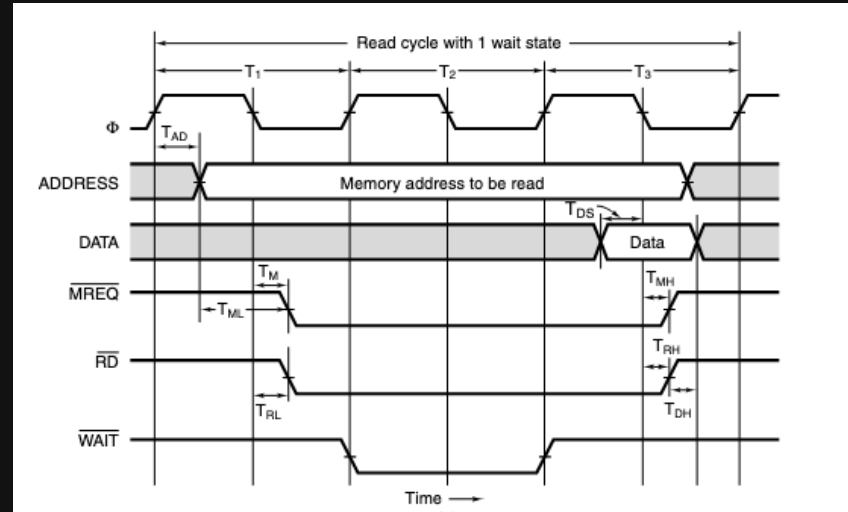
$\overline{MREQ}$ means memory is being accessed, and

$\overline{RD}$ means it's a read operation

Since our memory takes 15 ns to read, it can't provide the requested data during T_2 .

4. So the $\overline{WAIT}$ line is asserted at the start of T_2

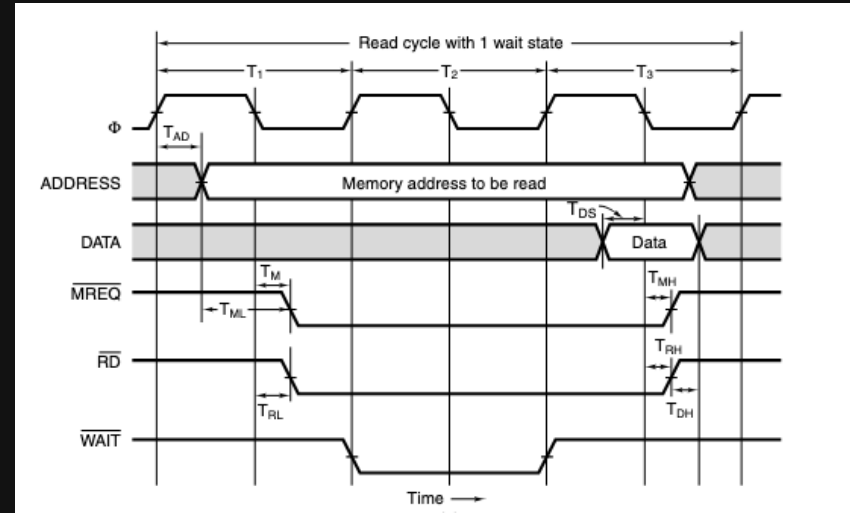
In our example, one clock cycle is in the $\overline{WAIT}$ state because the memory is too slow



Synchronous

Order of operations in this bus

5. At the start of T_3 , $\overline{WAIT}$ is deasserted, because the bus controller is sure that the memory has the data
6. During the first half of T_3 , the memory places the requested data on the data bus
7. On the falling edge of T_3 , the CPU strobcs (reads) the data lines and stores it in a register
8. The CPU then deasserts $\overline{MREQ}$ and $\overline{RD}$, and the bus cycle is complete



Synchronous

This timing chart shows the signals max and min times each operation would take, and is created by the BUS designers to be used by system designers

Symbol	Parameter	Min	Max	Unit
T_{AD}	Address output delay		4	nsec
T_{ML}	Address stable prior to $\overline{MREQ}$	2		nsec
T_M	$\overline{MREQ}$ delay from falling edge of Φ in T_1		3	nsec
T_{RL}	$\overline{RD}$ delay from falling edge of Φ in T_1		3	nsec
T_{DS}	Data setup time prior to falling edge of Φ	2		nsec
T_{MH}	$\overline{MREQ}$ delay from falling edge of Φ in T_3		3	nsec
T_{RH}	$\overline{RD}$ delay from falling edge of Φ in T_3		3	nsec
T_{DH}	Data hold time from negation of $\overline{RD}$	0		nsec

And with this information, we can say that at the worst case, it takes $25 - 4 - 2 = 19$ ns to read a word from memory

Where 10ns would mean responding on T_3 the same way 19ns would. But 20ns would mean responding on T_4

Synchronous

Using our 100 MHz clock (10 ns cycle time):

Memory access time	Bus cycles needed	Wait states	Effective read time
5 ns (very fast SRAM)	2	0	20 ns
15 ns (our example)	3	1	30 ns
25 ns (slow DRAM)	4	2	40 ns
35 ns (very slow device)	5	3	50 ns

The general rule: the CPU always rounds *up* to the next whole clock cycle, since it can't sample data mid-cycle. This is precisely the rigidity that motivated asynchronous buses.

Asynchronous

Synchronous buses are easier to work with, but they need to work in multiples of the bus clock cycle time.

For example, if the CPU and memory are able to complete a transfer in 3.1 cycles, they would have to wait until the 4th cycle to complete it.

Worse is that once a bus cycle is chosen, and things are built to use it, future improvements become difficult.

For example, imagine if new memories with an access time of 8 ns were invented instead of 15. This would mean it would take only 2 bus cycles to read a word, removing wait time

But if even newer memories with 4 ns access times were invented, it would still take 2 bus cycles because the minimum is 2 cycles

Asynchronous

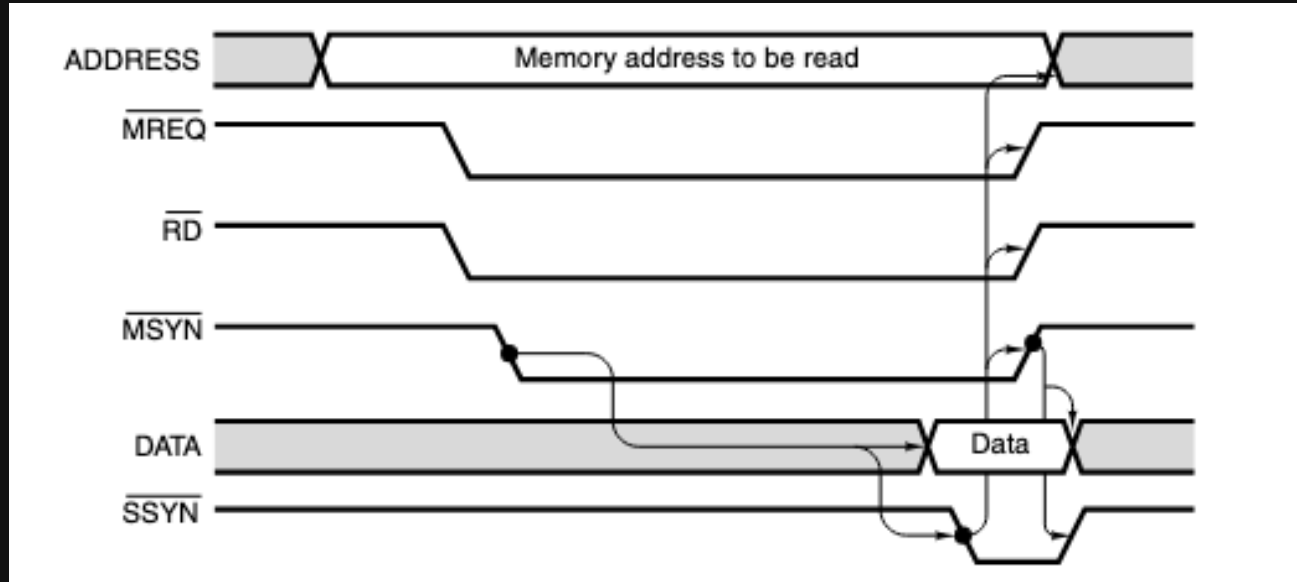
Suppose a synchronous bus has a fixed 10 ns cycle time and memory access takes only 4 ns:

- The transfer *could* complete in just over 4 ns, but the bus protocol still forces it to wait for the next full clock edge, so it takes a full 10 ns (2 cycles minimum, due to setup/hold requirements), wasting roughly 6 ns of the memory's actual speed every single access
- Across millions of memory accesses per second, this wasted time adds up to a significant fraction of total performance left on the table

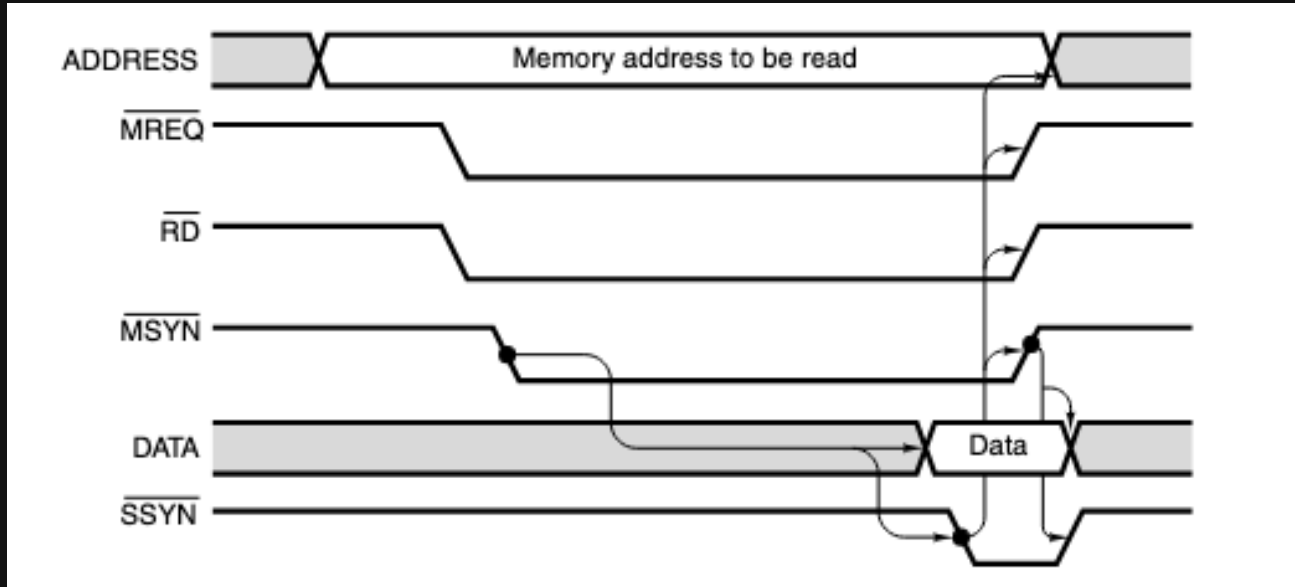
An asynchronous bus has no such floor: once the memory asserts $\overline{SSYN}$ as soon as data is ready, the CPU can accept it almost immediately, so speed improvements in memory translate *directly* into speed improvements for the whole system, with no clock-imposed minimum.

Asynchronous

This led to the development of asynchronous buses, where instead of a master clock, it types everything with the signals $\overline{MSYN}$ and $\overline{SSYN}$. Meaning Master and Slave Synchronize respectively



Asynchronous



1. Where to read data, it would assert the address and control lines as usual, then assert the $\overline{MSYN}$ line
2. And once the data is ready, the memory would assert $\overline{SSYN}$ to inform the CPU that the data is ready
3. The PC then deasserts $\overline{MSYN}$ and the other control line to inform the memory that it has accepted the data
4. The $\overline{SSYN}$ would then react and deassert itself

Asynchronous

This is called a **full handshake**

Which consists of:

1. $\overline{MSYN}$ is asserted
2. $\overline{SSYN}$ is asserted in response
3. $\overline{MSYN}$ is deasserted in response
4. $\overline{SSYN}$ is deasserted in response

This is faster, but because of how much easier synchronous buses are to design many Pcs still run in that system.

Bus Arbitration

We have assumed only one bus master so far.

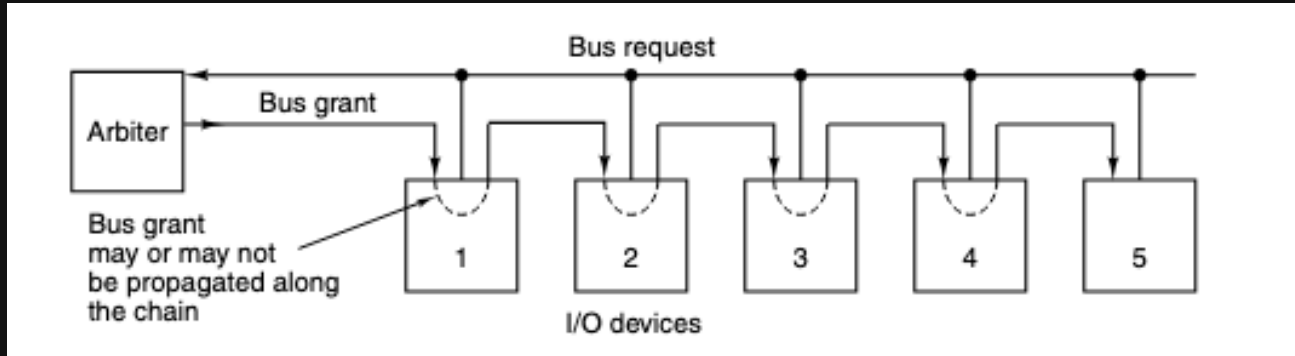
But in reality, there are often multiple bus masters, such as the I/O and the coprocessors' controller.

And the question "What happens if two or more devices all want to become bus master at the same time?" is called *bus arbitration*

Centralized Arbitration

Bus arbitration can be done in two ways, **centralized** or **distributed**.

In a centralized system, a single bus determines who goes next, and it's mostly implemented into the CPU.



It contains a single wired-OR that can be used for one or more devices to request at any time, then it grants the bus to one of them through the grant line.

And each device checks if they were the ones to send a request, and if not, they send it to the next one, until it finds the one that sent the request. This is called *daisy-chaining*

This means that the device closest to the CPU has the highest priority, and the one furthest away has the lowest

Centralized Arbitration

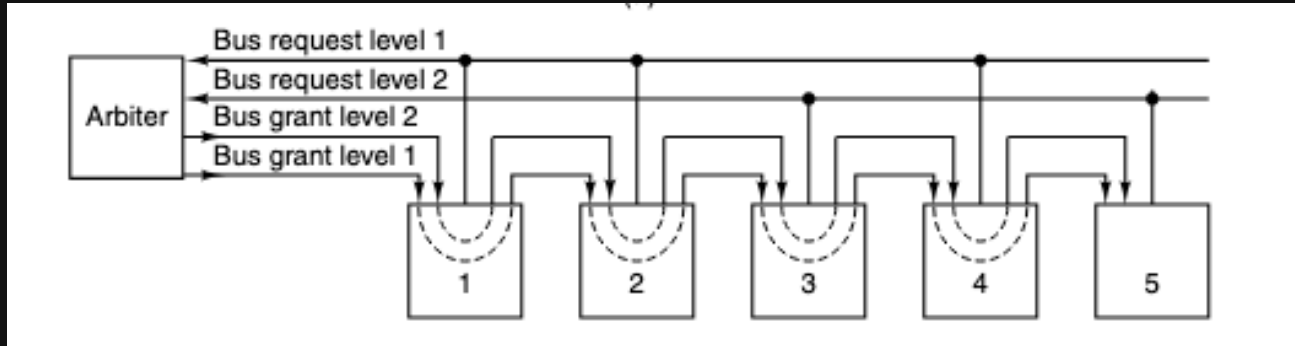
Suppose devices A, B, and C are daisy-chained in that order (A closest to the CPU, C furthest):

1. Both B and C assert the shared *request* line at roughly the same time
2. The arbiter (in the CPU) sees a request and asserts the *grant* line, which enters the chain at A
3. A checks: "Did I request the bus?" No - it passes the grant signal on to B
4. B checks: "Did I request the bus?" Yes - it takes the grant, becomes bus master, and does **not** forward the grant signal further down the chain
5. C never receives the grant this cycle, even though it also requested - it must wait for the next arbitration round

No matter how many times C requests, B will always win if it also has a pending request, simply because of its physical position closer to the CPU.

Centralized Arbitration

To get around implicit priorities, some systems use multiple priority levels, and if multiple priority levels are requested, the arbiter issues a grant only on the highest priority level.



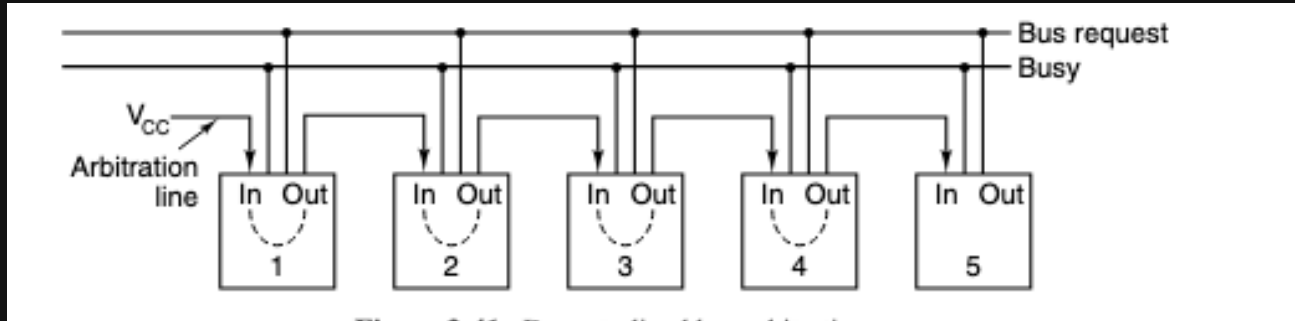
Some arbiters also have a third line called the acknowledgment line, Where the device that was granted the bus asserts this line to inform the arbiter that it has accepted the grant. Which frees up the request line for other devices to use. Essentially making requests non-blocking

Distributed Arbitration

Decentralized arbitration is also possible

One way of doing this uses 3 lines, with the first one being a wired-OR line representing the *bus request* line, the second one being another wired-OR line called *busy*, which is asserted by the current bus master.

And the third line is a daisy-chained line that's tied to the power supply and is used as the bus arbiter



To acquire the bus, a device asserts the request line, then waits for the arbitration line to reach it, then it asserts the busy line to inform other devices that it has acquired the bus

Where the leftmost device has the highest priority, and the rightmost device has the lowest. It's cheaper, faster, and not subject to arbiter failure, but less flexible.

Distributed Arbitration

The tradeoffs mirror the centralized case, but shift the complexity from a single dedicated arbiter chip to logic distributed across every device:

	Centralized	Distributed
Single point of failure	Yes - arbiter chip must work	No - no single arbiter to fail
Extra hardware needed	One arbiter unit	Small arbitration logic in <i>every</i> device
Flexibility (priority schemes)	Easy to add priority levels	Harder to change once wired
Typical use	Most modern buses (PCI, memory buses)	Some legacy minicomputer buses (e.g. Multibus, VME)

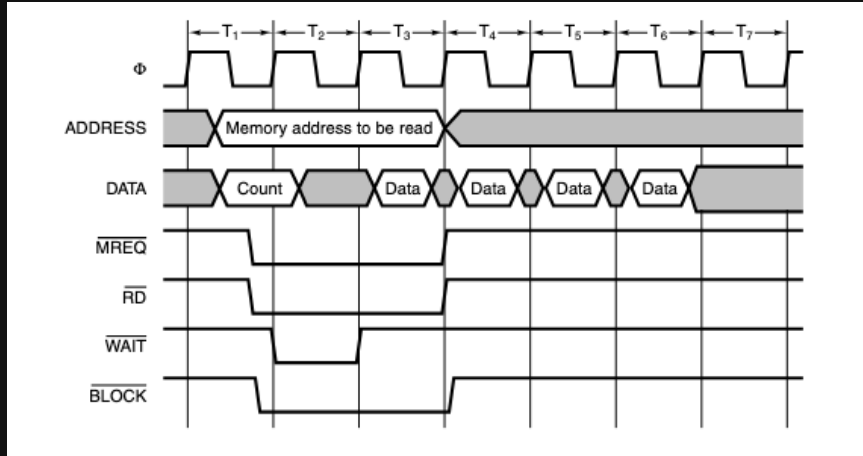
Both schemes still rely on the same underlying idea: some agreed-upon ordering (physical position on the chain) is used to break ties when multiple devices want the bus at once.

Bus Operation

Other Operations on the bus

The only operation we've discussed so far is reading from memory. There exists other types of operations and bus cycles.

For example, fetching an entire cache line



By putting the word count at the start of the line, and the addition of a ***BLOCK*** control line, the CPU can fetch an entire cache line in 6 bus cycles instead of 12

Bus Operation

A normal (non-block) read of n words needs 2 bus cycles per word: one to send the address, one to receive the data

So a 6-word cache line takes $6 \times 2 = 12$ cycles.

With a **block transfer**:

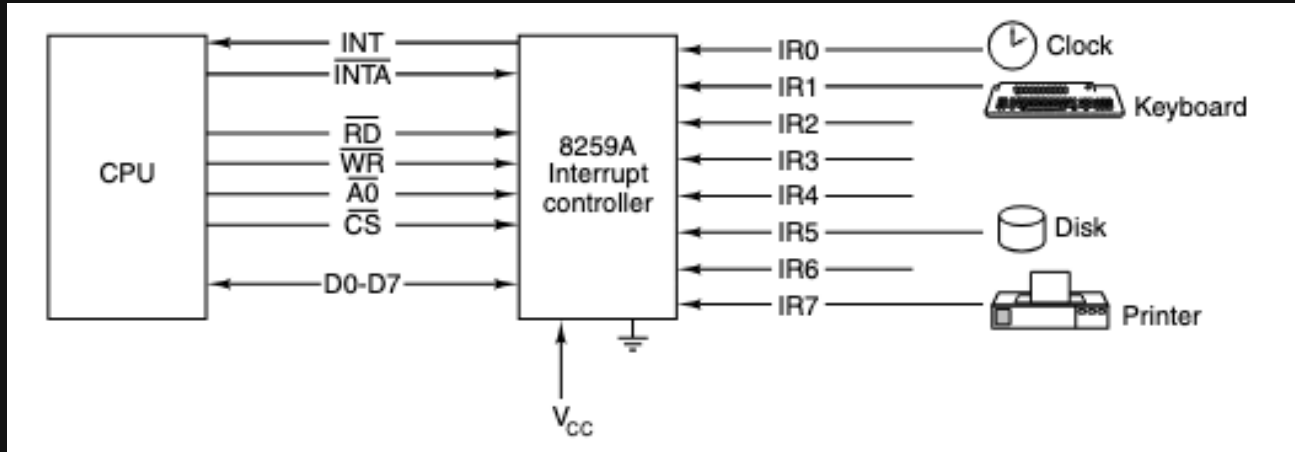
1. The address is sent **once**, along with a word count
2. The memory then streams back consecutive words, one per cycle, without the CPU needing to re-send an address each time

So the cost becomes 1 (address) + 6 (data) = 7, or as low as 6 if the count itself is piggybacked on the same cycle as the address. This is the same principle behind *burst mode* transfers in modern DRAM and cache-to-memory transfers.

Bus Operation

Multiprocessor systems often need to have a read-modify-write cycle, where a CPU reads a word from memory, modifies it, then writes it back without releasing the bus.

And another kind of bus cycle is for handling interrupts



Where devices send to the interrupt controller, which then sends an interrupt request to the CPU, and the controller decides which device gets to interrupt the CPU next

Bus Operation

Read-modify-write, used to implement atomic operations like `test-and-set` for synchronization primitives (e.g. mutex locks):

1. CPU becomes bus master and reads a memory word
2. CPU modifies the value internally
3. CPU writes the new value back, all **without releasing the bus** in between

This guarantees no other bus master (like a second CPU core) can sneak in a conflicting read or write between steps 1 and 3, which is essential for correct multiprocessor locking.

Interrupt vectoring, used so the CPU knows *which* handler to run:

1. A device asserts its interrupt line to the interrupt controller
2. The controller asserts $\overline{INT}$ to the CPU
3. The CPU responds with an **interrupt acknowledge** cycle
4. The controller places a small number (the *interrupt vector*) on the data bus, identifying which device interrupted
5. The CPU uses that vector to look up and jump to the correct interrupt handler

This avoids the CPU having to poll every device to figure out who caused the interrupt.